Introducción

El objetivo de esta práctica fue desarrollar un circuito controlado mediante un Arduino Uno que permitiera controlar la dirección y velocidad de dos motores de corriente directa (DC), así como detectar obstáculos mediante un sensor ultrasónico HC-SR04. Como complemento, se incorporó un buzzer que emite una señal acústica cuando el sistema se encuentra en reversa y detecta un objeto dentro del rango establecido. La finalidad del proyecto fue integrar el control de motores con la medición de distancias y una señal de advertencia, simulando una función de asistencia para maniobras en reversa.

Metodología

Materiales:

- Arduino Uno R3.
- Protoboard.
- Controlador de motores L293D.
- Dos motores de corriente directa (DC).
- Sensor ultrasónico HC-SR04
- Buzzer
- Resistencia de 220 Ω
- Cables jumper.
- Computadora para alimentar el Arduino.
- Fuente de alimentación externa para los motores.

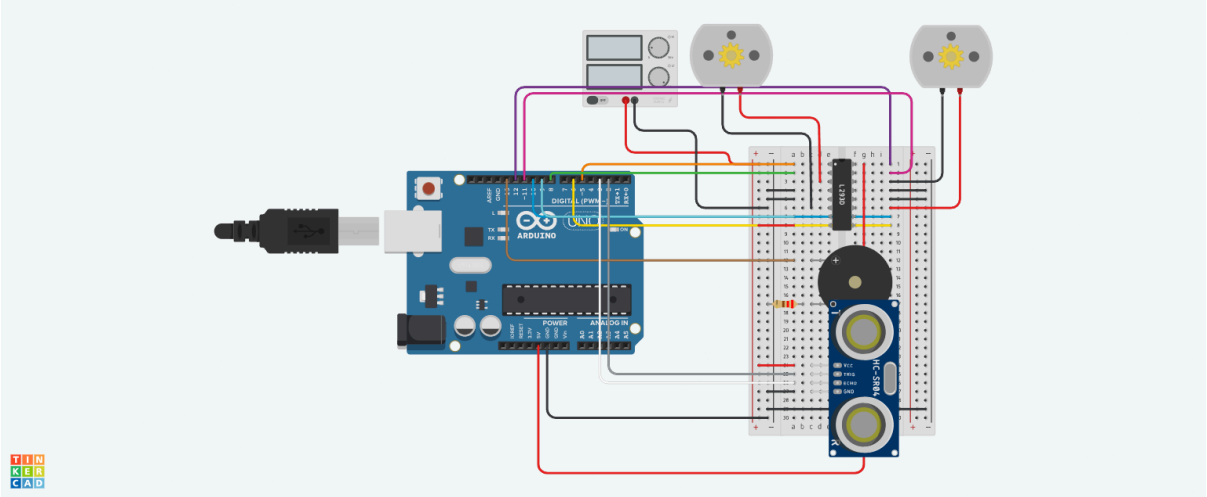
El Arduino Uno fue utilizado como el controlador principal del circuito, encargado de ejecutar el programa, controlar la dirección y velocidad de los motores, y procesar las mediciones del sensor ultrasónico. Debido a que los pines del Arduino no están diseñados para suministrar directamente la corriente requerida como controlador de potencia. Este componente permite cambiar la dirección de giro de los motores mediante señales digitales y regular su velocidad utilizando señales PWM.

Los dos motores se conectaron a las salidas del L293D, mientras que las entradas de control y habilitación del circuito integrado se conectaron a pines digitales del Arduino. Los pines de habilitación EN1 y EN2 se utilizaron para controlar la velocidad de los motores mediante señales PWM, mientras que las entradas IN1, IN2, IN3 e IN4 permitieron establecer su dirección de giro.

Por otra parte, se incorporó un sensor ultrasóinco HC-SR04 para medir la distancia entre el circuito y un posible obstaculo. Este sensor utiliza un transmisor y un receptor ultrasóinco: el primero emite un pulso sonoro y el segundo detecta su eco después de reflejarse en un

objeto. El Arduino calcula la distancia a partir del tiempo que tarda el eco en regresar. Para ello, se conectaron los pines trig y echo del sensore a los pines digitales 2 y 3, respectivamente.

Finalmente, se conectó un buzzer al pin digital 13 del Arduino para producir una señal acústica cuando los motores se encuentran en reversa y el sensor detecta un objeto dentro del rango programado de 350 cm. La frecuencia de los pitidos aumenta conforme disminuye la distancia al objeto, proporcionando una advertencia más frecuente cuando el obstáculo está más cerca. Todos los componentes comparten una referencia de tierra común para permitir el funcionamiento adecuado del circuito.



El objetivo de esta práctica fue desarrollar un circuito controlado mediante un Arduino Uno que permitiera controlar la dirección y velocidad de dos motores de corriente directa (DC), así como detectar obstáculos mediante un sensor ultrasónico HC-SR04. Como complemento, se incorporó un buzzer que emite una señal acústica cuando el sistema se encuentra en reversa y detecta un objeto dentro del rango establecido. La finalidad del proyecto fue integrar el control de motores con la medición de distancias y una señal de advertencia, simulando una función de asistencia para maniobras en reversa.

En el nuevo circuito, el Arduino se utilizó como controlador principal para establecer la comunicación inalámbrica con el módulo Bluetooth HC-05 y controlar el LED. El módulo Bluetooth fue conectado mediante comunicación serial utilizando los pines digitales 10 y 11 del Arduino. El pin 10 se utilizó como TX y el pin 11 como RX . Debido a que el nivel lógico de entrada del HC-05 es de aproximadamente 3.3 V, la conexión hacia su pin RX se realizó mediante un divisor de voltaje formado por dos resistencias (la resistencia de 1kΩ desde GND y la resistencia de 2kΩ desde el pin 11), reduciendo el voltaje proveniente del Arduino a un nivel adecuado para el módulo. Por otra parte, el pin 13 del Arduino se utilizó para controlar el LED, conectado en serie con una resistencia de 110 Ω para limitar la corriente y proteger el componente. Finalmente, se estableció una conexión común a tierra entre los componentes del circuito, permitiendo compartir una referencia eléctrica y garantizar el correcto funcionamiento de la comunicación y del LED.

Código utilizado

CodigoDosMotoresSensorUltrasonic.ino

Este algoritmo fue desarrollado con ayuda de inteligencia artificial

```
int EN1 = 5;
int IN1 = 8;
int IN2 = 9;

int EN2 = 6;
int IN3 = 10;
int IN4 = 11;

int trigPin = 2;
int echoPin = 3;
int buzzer = 13;

bool vaAtras = false;

unsigned long tiempoFase = 0;
unsigned long tiempoBuzzer = 0;
unsigned long duracion;

unsigned int distancia;
int intervalo;
```

```
21
22  void setup() {
23    pinMode(EN1, OUTPUT);
24    pinMode(IN1, OUTPUT);
25    pinMode(IN2, OUTPUT);
26
27    pinMode(EN2, OUTPUT);
28    pinMode(IN3, OUTPUT);
29    pinMode(IN4, OUTPUT);
30
31    pinMode(trigPin, OUTPUT);
32    pinMode(echoPin, INPUT);
33    pinMode(buzzer, OUTPUT);
34
35    moverAdelante();
36    tiempoFase = millis();
37  }

39  void loop() {
40    if (millis() - tiempoFase >= 2000) {
41      tiempoFase = millis();
42
43      if (vaAtras) {
44        moverAdelante();
45      } else {
46        moverAtras();
47      }
48    }
49
50    actualizarBuzzer();
51  }

53  void moverAdelante() {
54    digitalWrite(IN1, HIGH);
55    digitalWrite(IN2, LOW);
56    digitalWrite(IN3, HIGH);
57    digitalWrite(IN4, LOW);
58
59    analogWrite(EN1, 120);
60    analogWrite(EN2, 120);
61
62    vaAtras = false;
63  }
```

```
void moverAtras() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  analogWrite(EN1, 120);
  analogWrite(EN2, 120);

  vaAtras = true;
}

void actualizarBuzzer() {
  distancia = medirDistancia();

  // El buzzer solo funciona al ir en reversa
  if (!vaAtras || distancia == 0 || distancia > 350) {
    noTone(buzzer);
    return;
  }

  intervaloBuzzer(distancia);
}

void intervaloBuzzer(int cm) {
  int intervalo = map(constrain(cm, 5, 350),
                      5, 350, 100, 800);

  if (millis() - tiempoBuzzer >= (unsigned long)intervalo) {
    tone(buzzer, 500, 50);
    tiempoBuzzer = millis();
  }
}
```

```
99  unsigned int medirDistancia() {
100   digitalWrite(trigPin, LOW);
101   delayMicroseconds(2);
102
103   digitalWrite(trigPin, HIGH);
104   delayMicroseconds(10);
105
106   digitalWrite(trigPin, LOW);
107
108   duracion = pulseIn(echoPin, HIGH, 25000);
109
110   if (duracion == 0) {
111     return 0;
112   }
113
114   return (unsigned int)(duracion * 0.0343 / 2);
115 }
```

Explicación del código

```
1   int EN1 = 5;
2   int IN1 = 8;
3   int IN2 = 9;
4
5   int EN2 = 6;
6   int IN3 = 10;
7   int IN4 = 11;
8
9   int trigPin = 2;
10  int echoPin = 3;
11  int buzzer = 13;
12
13  bool vaAtras = false;
14
15  unsigned long tiempoFase = 0;
16  unsigned long tiempoBuzzer = 0;
17  unsigned long duracion;
18
19  unsigned int distancia;
20  int intervalo;
```

Al principio del programa se declaran los pines del Arduino utilizados para controlar los motores, el sensor ultrasónico y el buzzer. Los pines EN1 y EN2 corresponden a las entradas de habilitación del controlador L293D, mientras que IN1, IN2, IN3 e IN4 permiten establecer la dirección de giro de los dos motores.

También se declara la variable booleana vaAtras, que indica si el programa ha configurado los motores para retroceder. Inicialmente, su valor es false, ya que el circuito comienza avanzando. Asimismo, se utilizan variables para registrar el tiempo transcurrido, almacenar

la duración del pulso ultrasónico, guardar la distancia medida y controlar el intervalo entre pitidos.

```
21
22  void setup() {
23    pinMode(EN1, OUTPUT);
24    pinMode(IN1, OUTPUT);
25    pinMode(IN2, OUTPUT);
26
27    pinMode(EN2, OUTPUT);
28    pinMode(IN3, OUTPUT);
29    pinMode(IN4, OUTPUT);
30
31    pinMode(trigPin, OUTPUT);
32    pinMode(echoPin, INPUT);
33    pinMode(buzzer, OUTPUT);
34
35    moverAdelante();
36    tiempoFase = millis();
37  }
```

Dentro de la función setup(), se configuran los pines correspondientes como entradas o salidas. Después, se llama a la función moverAdelante() para establecer la dirección inicial de los motores y se registra el tiempo de inicio mediante millis().

```
39  void loop() {
40    if (millis() - tiempoFase >= 2000) {
41      tiempoFase = millis();
42
43      if (vaAtras) {
44        moverAdelante();
45      } else {
46        moverAtras();
47      }
48    }
49
50    actualizarBuzzer();
51  }
```

La función loop() se ejecuta continuamente mientras el Arduino permanece encendido. En ella se comprueba si han transcurrido dos segundos desde el último cambio de dirección. Cuando esto ocurre, se invierte el estado de los motores: si estaban avanzando, pasan a reversa; si estaban retrocediendo, vuelven a avanzar.

Posteriormente, se llama a la función actualizarBuzzer(), que se ejecuta independientemente de la dirección de los motores. Esto permite que el sensor continúe midiendo distancias durante ambas fases, aunque el buzzer solo se active cuando se cumple la condición de reversa.

```
53  void moverAdelante() {
54    digitalWrite(IN1, HIGH);
55    digitalWrite(IN2, LOW);
56    digitalWrite(IN3, HIGH);
57    digitalWrite(IN4, LOW);
58
59    analogWrite(EN1, 120);
60    analogWrite(EN2, 120);
61
62    vaAtras = false;
63  }
```

La función moverAdelante() configura las entradas del L293D de manera que ambos motores giren en la dirección definida como avance. Para ello, establece IN1 e IN3 en HIGH e IN2 e IN4 en LOW. Posteriormente, utiliza analogWrite() para aplicar un valor PWM de 120 a las entradas de habilitación EN1 y EN2. Este valor regula la potencia promedio suministrada a los motores y, por tanto, su velocidad.

```
65  void moverAtras() {
66    digitalWrite(IN1, LOW);
67    digitalWrite(IN2, HIGH);
68    digitalWrite(IN3, LOW);
69    digitalWrite(IN4, HIGH);
70
71    analogWrite(EN1, 120);
72    analogWrite(EN2, 120);
73
74    vaAtras = true;
75  }
```

Por otro lado, la función moverAtras() invierte los estados de las entradas de dirección: IN1 e IN3 pasan a LOW, mientras que IN2 e IN4 pasan a HIGH. De esta manera, ambos motores giran en sentido contrario. Se mantiene el mismo valor PWM de 120 para conservar la misma configuración de velocidad en ambas direcciones.

Además, cada función actualiza la variable vaAtras. Esta variable es importante porque permite que el resto del programa identifique la dirección ordenada a los motores sin necesidad de volver a comprobar individualmente las señales de control.

```
77  void actualizarBuzzer() {
78    distancia = medirDistancia();
79
80    // El buzzer solo funciona al ir en reversa
81    if (!vaAtras || distancia == 0 || distancia > 350) {
82      noTone(buzzer);
83      return;
84    }
85
86    intervaloBuzzer(distancia);
87  }
```

La función actualizarBuzzer() obtiene continuamente la distancia mediante medirDistancia(). Después, comprueba tres condiciones: que los motores estén configurados para retroceder, que la medición sea válida y que el objeto se encuentre a una distancia de 350 cm o menos.

Si alguna de estas condiciones no se cumple, se ejecuta noTone() para mantener el buzzer apagado. En caso contrario, se llama a la función intervaloBuzzer() y se utiliza la distancia medida para determinar la frecuencia temporal de los pitidos.

```
89  void intervaloBuzzer(int cm) {
90    int intervalo = map(constrain(cm, 5, 350),
91                        5, 350, 100, 800);
92
93    if (millis() - tiempoBuzzer >= (unsigned long)intervalo) {
94      tone(buzzer, 500, 50);
95      tiempoBuzzer = millis();
96    }
97  }
```

La función intervaloBuzzer() emplea la función map() para establecer una relación aproximadamente lineal entre la distancia y el intervalo de tiempo entre pitidos. Dentro del rango programado, una distancia cercana a 350 cm produce intervalos de aproximadamente 800 milisegundos, mientras que una distancia de 5 cm o menos produce intervalos de aproximadamente 100 milisegundos.

Cuando transcurre el intervalo calculado, la función tone() genera un sonido de 500 Hz durante 50 milisegundos. Posteriormente, se actualiza la variable tiempoBuzzer para determinar cuándo debe producirse el siguiente pitido. De esta manera, el sistema emite advertencias acústicas progresivamente más frecuentes conforme el objeto se aproxima.

```
 99  unsigned int medirDistancia() {
100    digitalWrite(trigPin, LOW);
101    delayMicroseconds(2);
102
103    digitalWrite(trigPin, HIGH);
104    delayMicroseconds(10);
105
106    digitalWrite(trigPin, LOW);
107
108    duracion = pulseIn(echoPin, HIGH, 25000);
109
110    if (duracion == 0) {
111      return 0;
112    }
113
114    return (unsigned int)(duracion * 0.0343 / 2);
115  }
```

La función medirDistancia() controla el sensor ultrasónico HC-SR04. Primero, establece el pin TRIG en LOW durante un breve intervalo para estabilizar la señal. Después, genera un

pulso HIGH de 10 microsegundos, lo que provoca que el sensor emita una señal ultrasónica.

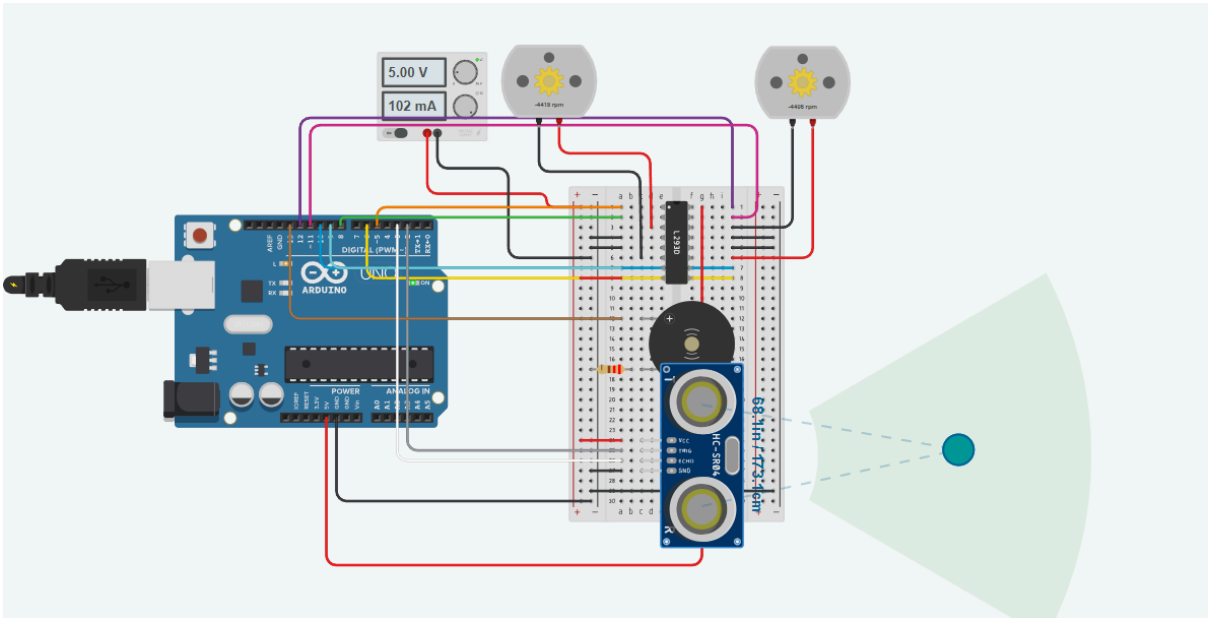
A continuación, se utiliza la función pulseIn() para medir el tiempo durante el cual la señal ECHO permanece en HIGH. Este tiempo corresponde al recorrido de ida y vuelta de la onda ultrasónica entre el sensor y el objeto.

La distancia se calcula multiplicando la duración del pulso por la velocidad aproximada del sonido en el aire, de 0.0343 cm/µs, y dividiendo el resultado entre dos para considerar únicamente la distancia de ida.

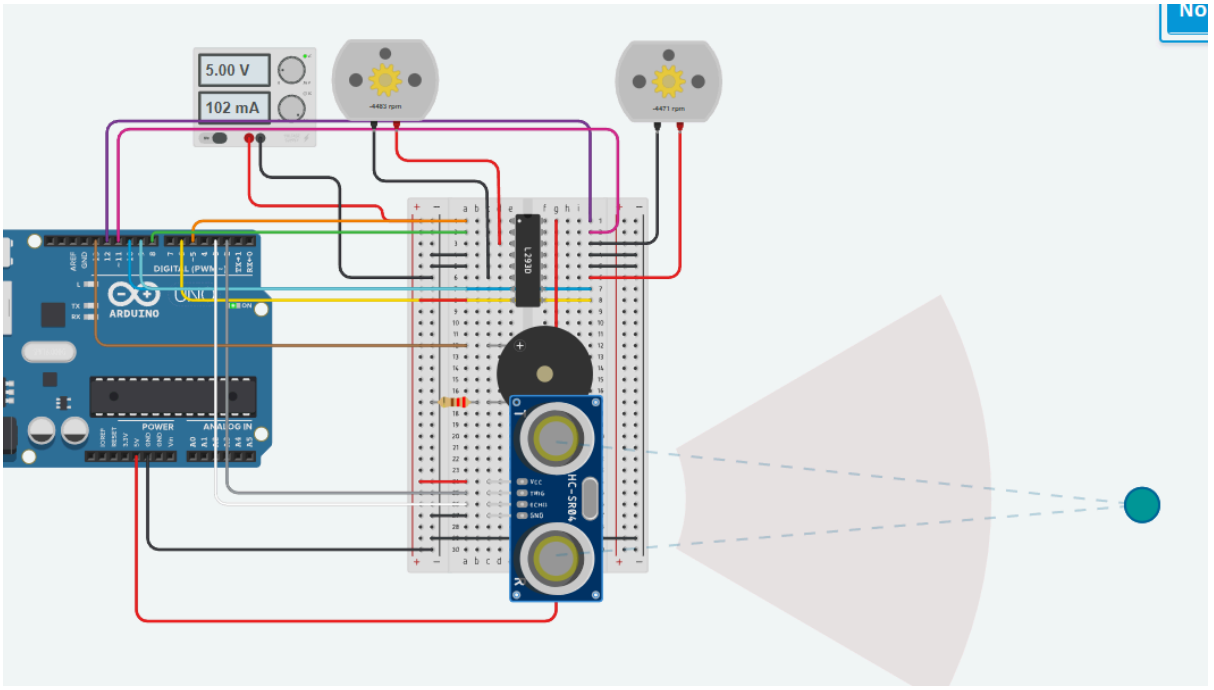
Si no se recibe un eco dentro del tiempo límite establecido, la función devuelve un valor de cero. Este resultado se interpreta como una medición no válida y evita que el programa trate esa lectura como una distancia real.

Resultados

- Al entrar en la fase de reversa y el objeto estar en dentro del rango del sensor, se detectara y causará un sonido con el buzzer:

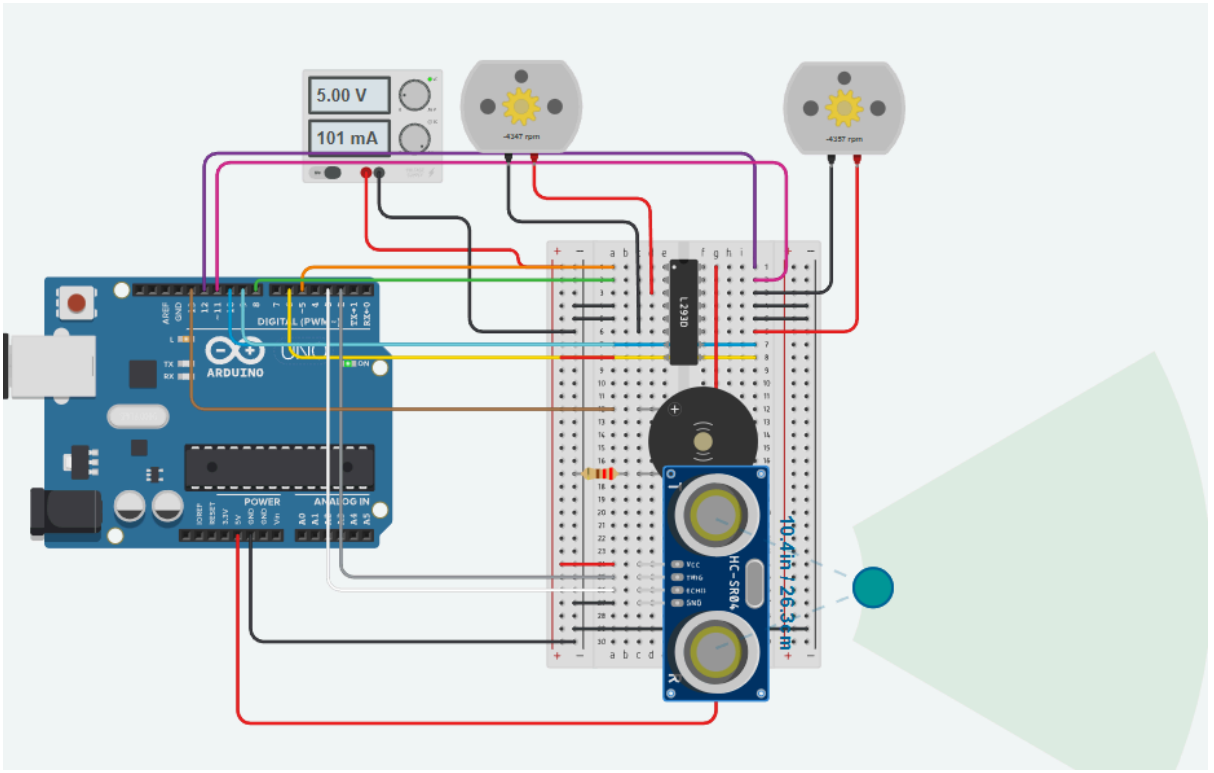


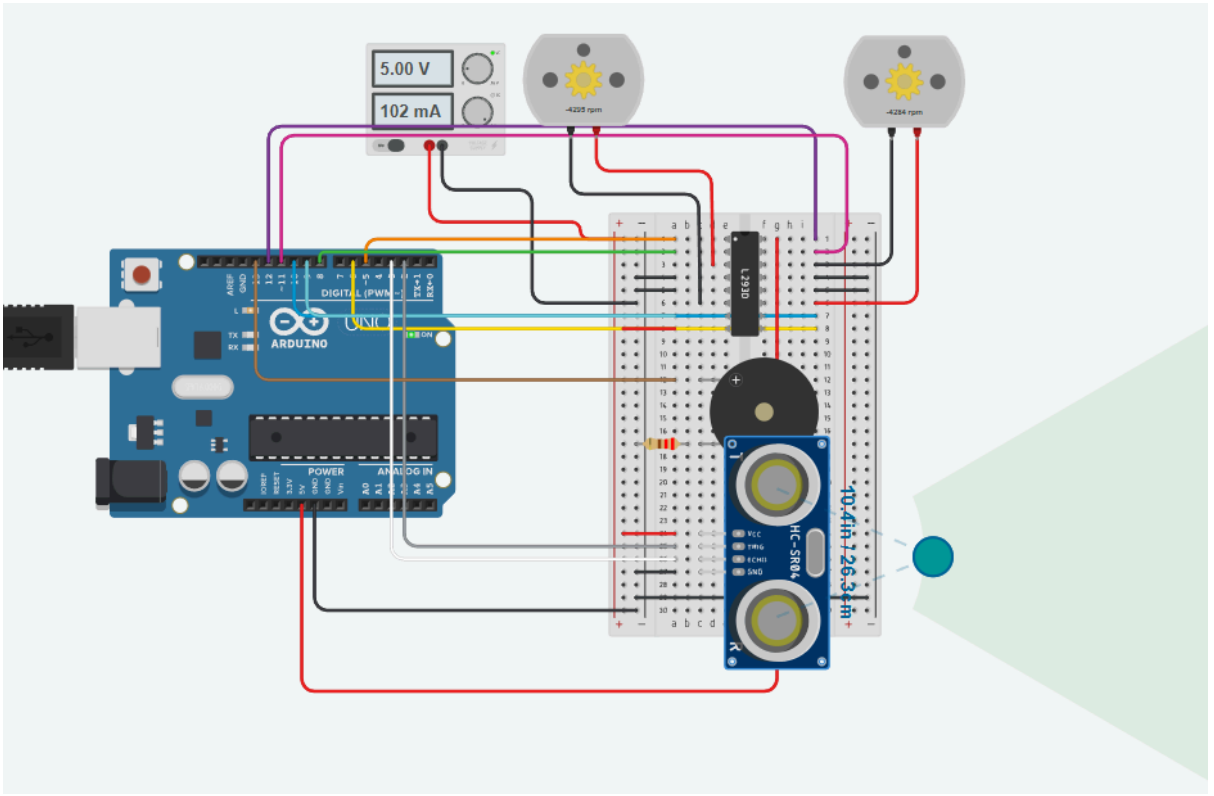
- A pesar de que este en fase de reversa, como el objeto no entra en la distancia del sensor ultrasónico, no se emitirá ningun sonido con el buzzer:



.

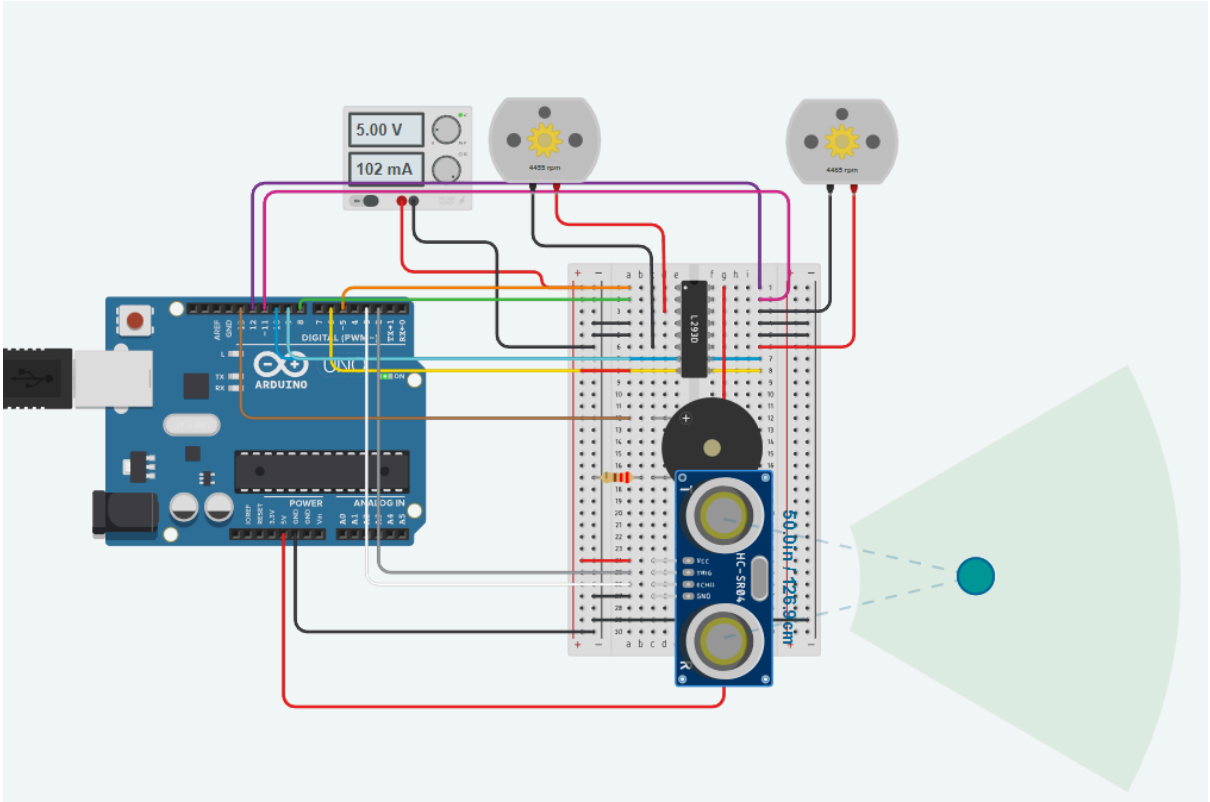
- Al disminuir la distancia, el buzzer emitira una frecuencia con mayor rapidez (esto se puede apreciar mejor en el repositorio, puesto que hay evidencia de video):





.

- A pesar de que el objeto se encuentre en el rango del sensor ultrasónico, no se emitra ninguna frecuencia del buzzer, ya que el algoritmo no esta en fase de reversa:



(Al igual que en el circuito pasado, hemos definido la fase de reversa donde los motores muestran revoluciones por minuto negativas).

Conclusión

El desarrollo de este circuito permitió reintegrar diferentes componentes electrónicos para implementar un sistema de control de motores con detección de obstáculos. El Arduino Uno coordinó el funcionamiento del controlador L293D, el sensor ultrasónico HC-SR24 y el buzzer mediante un programa que combina señales digitales, control PWM y medición del tiempo.

El uso del L293D permitió controlar la dirección de giro de los dos motores sin conectarlos directamente a las salidas del Arduino. Asimismo, la aplicación de señales PWM permitió regular la velocidad de los motores mediante el valor establecido en el programa. Por otro lado, el sensor ultrasónico permitió calcular la distancia a un objeto a partir del tiempo de retorno de una onda sonora.

Finalmente, la incorporación del buzzer permiitió desarrollar una advertencia acústica condicionada por la dirección de los motores y la distancia detectada. Esta práctica reforzó los conocimientos sobre programación de microcontroladores, control de actuadores, adquisición de datos de sensores y coordinación de diferentes componentes electrónicos de un mismo sistema.

Referencias bibliográficas:

- OpenAI. (2026). ChatGPT (GPT-6) [Modelo de lenguaje grande]. https://chatgpt.com/